

OMNIVERSAL ARCHITECT LABS

Adversarial Cross-Review Protocol and Reusable Template

A replicable method for reviewing OAL AI-collaboration documents without seeking consensus, with a pilot-execution standard, a multi-AI comparison procedure, and cost tiers

Document ID	OAL-ARP-TEMPLATE-CLAUDE-001
OAL version	v0.1 (proposed method; derived from the End Cycle Laboratory cross-review OAL-ECL-XR-PERPLEXITY-001 v0.2)
Date	October 1, 2026
Prepared by	Claude (Anthropic)
Prepared for	Nathan Lewis II, Architect of Record
Status	PROPOSED. NOT CANON. NOT ONESTAMP-RATIFIED. PENDING ARCHITECT REVIEW.
Format	Delivered as PDF and DOCX for GitHub publication

Speculative worldbuilding and research-method document. Nothing here models any real person's grief, depression, burnout, disability, or suicidal thinking. In the US, anyone in crisis can call or text 988.

Contents

1. Executive summary
 2. Versioning record and independence declaration
 3. Mission statement and directive
 4. Overview
 5. Principles
 6. Roles
 7. The seven-phase procedure
 8. Per-document review form (the core template)
 9. Discrepancy register template
 10. Pilot execution standard
 11. Claim labels
 12. Multi-AI replication and comparison protocol
 13. Cost tiers: Lite, Standard, Full
 14. Quality gates and known failure modes
 15. Preparer's feedback on the Perplexity cross-review and pilot
 16. Open questions for the Architect and team
- Appendix A. Blank review form
- Appendix B. Copy-ready reviewer prompt
- Appendix C. Naming and registry scheme
- Appendix D. Pre-run checklist for simulations

1. Executive summary

This document turns what Perplexity Computer did in OAL-ECL-XR-PERPLEXITY-001 (an adversarial cross-review of five End Cycle Laboratory designs, plus three small executed simulations) into a procedure that can be applied to any OAL framework, document set, or AI-generated design. The goal is not agreement. The goal is to find holes, weaknesses, omissions, unasked questions, and unperformed tasks, and to convert them into specific, assignable improvements.

The protocol has seven phases: scope, inventory, independent review, discrepancy registration, pilot execution, cross-AI comparison, and Architect decision. Each reviewing AI works independently from the same template; the Architect then compares versions. Disagreement is recorded, never averaged. Claims count as findings only if something was actually run. Anything one author both builds and judges is flagged as circular.

Because a full multi-AI pass is expensive, the protocol offers three tiers. Lite needs one reviewer and no code. Standard adds a second reviewer and one pilot. Full uses all six AIs, separated generator and detector roles, and a comparison pass. The preparer also gives specific feedback on the Perplexity review (Section 15): its strongest ideas are kept, and several weaknesses in its pilot code and its reporting are recorded so the template does not inherit them.

2. Versioning record and independence declaration

Field	Entry
Title	Adversarial Cross-Review Protocol and Reusable Template
Document ID	OAL-ARP-TEMPLATE-CLAUDE-001 (proposed; see Appendix C and Section 16, question 1)
Version	v0.1
Date	October 1, 2026
Preparer	Claude (Anthropic)
Status	PROPOSED. NOT CANON. NOT ONESTAMP-RATIFIED.
Supersedes	Nothing. Derived from OAL-ECL-XR-PERPLEXITY-001 v0.2.
Posture	Adversarial and non-consensus, including toward the document it derives from and toward Claude's own earlier work.

Independence declaration (inputs read)

- The Architect's message of October 1, 2026 describing the goal and the Perplexity cross-review.
- OAL-ECL-XR-PERPLEXITY-001 v0.2, PDF (read in full).
- ECL pilot results (results.json) and ECL Iteration 2 pilot simulations (pilot.py), read in full.
- The DOCX version of the cross-review was supplied but not separately opened; it is assumed to match the PDF. This is an assumption, not a verification.
- The five Iteration 1 documents themselves were NOT read. Everything this document says about them comes from the Perplexity review and is secondhand.
- This document was not independent of the Perplexity review. It is a derivative method document, and its Section 15 critique should be read with that in mind.

3. Mission statement and directive

Mission

The Adversarial Cross-Review Protocol exists to make OAL's generated work testable and correctable. It subjects every framework, outline, and design document to review by parties who are not trying to agree with it, and it converts each criticism into a recorded finding, a question, or a task.

Directive

- Review to find what is wrong, missing, circular, untested, or inconsistent, not to confirm what is good.
- Do not seek consensus. Record disagreement between reviewers side by side.
- Count a claim as a finding only when something was executed. Specification is not evidence.
- Treat agreement among AI systems as weak evidence when they received overlapping prompts.
- Every review ends in actions: a decision for the Architect, a test to run, or a records fix.
- Respect the Architect's source material as given, unless the Architect asks for it to be examined.

What the protocol is not

- Not a quality seal. A document that survives review has survived one review.
- Not canon. Nothing here is ONESTAMP-ratified.
- Not a requirement to run monthly. The tiers in Section 13 exist because the Architect has limited time and money.

4. Overview

OAL has produced a large body of documentation across several AIs, published on GitHub. The Perplexity cross-review showed that a large body of design can contain no executed test at all: five End Cycle Laboratory versions held roughly 54,000 words of design and zero results. It also showed that an independent reviewer can surface contradictions (24 registered), mislabeled stages, missing controls, and self-entrenching governance that the authors did not see.

This protocol captures that pattern in a form any AI, or any human on the team, can run. It has three layers:

- **Review layer:** a fixed form each reviewer fills in per document (Section 8).
- **Execution layer:** a standard for small pilot simulations, so results are reproducible and not self-confirming (Section 10).
- **Comparison layer:** a procedure for the Architect to compare independent reviews from several AIs without averaging them (Section 12).

5. Principles

- **Evidence over elegance.** A well-written framework with no run behind it is a proposal. Label it that way.
- **Separate who builds from who judges.** If one author builds the world and the detector, the result restates the author's assumptions.
- **Controls first.** Every test needs a no-change or no-intervention case. Without it, ordinary variation looks like a finding.

- **Both failure directions.** Test whether a rule fails by acting too much and by acting too little.
- **Price the procedure.** Review, delay, and process have costs. Report them next to the benefit.
- **Reviewers review themselves first.** Each reviewer critiques its own earlier work with the same severity as others'.
- **Preserve dissent.** Keep minority positions alive as named alternatives until evidence retires them.
- **Receive the Architect's material as given.** Reviews do not reinterpret or fact-check source material unless the Architect requests it. Any reading is labeled PROPOSAL.

6. Roles

Role	Does	Must not
Architect	Selects scope and tier; assigns roles; makes all DECISION items; compares reviews; may pause the process at any time.	Be required to approve every step (bottleneck).
Reviewer (one per AI)	Fills in the review form independently; reviews its own earlier work first.	Read other reviewers' output before finishing its own pass, unless the tier allows it and it declares that.
Generator	Builds a test world and records ground truth privately (sealed file or commit).	Also act as detector for that world.
Detector	Diagnoses the generator's observations blind and reports confidence.	See ground truth.
Red team	Attacks the detector's diagnosis and the reviewer's strongest claims.	Be the same AI as the one it attacks.
Registrar	Maintains IDs, crosswalks, discrepancy register, and version history. May be the Architect or a designated AI.	Resolve discrepancies. Only the Architect decides.

7. The seven-phase procedure

Phase 0. Scope

Architect names the topic, the document set, the tier, and the output deadline. Output: a one-page scope note. Stop rule: no review starts without a scope note.

Phase 1. Inventory

List every document under review: title, ID, preparer, version, date, claimed stage, actual stage delivered, approximate size. Note anything missing (as the Perplexity review noted Gemini's missing version). Output: source corpus table.

Phase 2. Independent review

Each reviewer fills in the Section 8 form for every document, its own first. Reviewers do not see each other's output. Output: one review document per reviewer.

Phase 3. Discrepancy registration

Collect conflicts between documents (stage models, IDs, definitions, rules, scores). Each gets an ID, the conflicting positions, why it matters, and a resolution path: TEST, DECISION, or HYGIENE. Output: discrepancy register (Section 9).

Phase 4. Pilot execution

Run the smallest tests that could prove the work wrong, following Section 10. Output: code, raw results, and a findings table with PASS, FAIL, MIXED, INDETERMINATE, or NOT RUN.

Phase 5. Cross-AI comparison

Architect (or Registrar) builds the comparison matrix of Section 12 from all reviewers' outputs. Output: comparison matrix and a list of convergences, divergences, and unique findings.

Phase 6. Decision and iteration

Architect resolves DECISION items, authorizes TEST items, assigns HYGIENE items, and issues the next-iteration commission. Output: decision log and next commission. Stop rule: nothing becomes canon here.

8. Per-document review form (the core template)

Each reviewer answers these questions for every document. Short, specific answers are better than long general ones. Each answer carries a claim label from Section 11.

#	Question	What a good answer contains
1	What does it contribute that no other document does?	Named ideas, instruments, or scenarios, with where they appear.
2	What does it get right?	Specific strengths worth keeping, so criticism does not erase them.
3	Where is it internally inconsistent?	Quotes or section references for each contradiction (own-rule violations count).
4	Where does it assume what it claims to test?	The claim, the assumption hiding inside it, and what test would separate them.
5	What is missing?	Controls, edge cases, failure directions, unasked questions, tasks never performed.
6	Can it be executed as written?	What a single AI in a single session could and could not run; what is undefined (thresholds, who decides).
7	How does it treat the Architect's source material?	Does it add meaning, premises, or quotes the Architect did not supply? Flag, do not rule on correctness.
8	Provenance and registry	ID, version, preparer line, stage claimed vs. delivered, inputs declared vs. inputs used.
9	Who could break it?	The strongest attack on the document from the opposite design philosophy.
10	Keep, change, quarantine	Three lists with reasons. Quarantine means held out of the next iteration until tested.

Summary row. Each review ends with one line per document: strongest feature, most serious weakness, and the single most useful next task.

9. Discrepancy register template

ID	Discrepancy	Positions (by document)	Why it matters	Resolution path
D-01	Short name	Doc A: ... Doc B: ...	One sentence	TEST / DECISION / HYGIENE plus the

ID	Discrepancy	Positions (by document)	Why it matters	Resolution path
				concrete step

- Number discrepancies D-01 onward per review set.
- Never resolve by averaging or splitting the difference.
- TEST: define the run and what outcome would favor each side before running it.
- DECISION: state the question in one sentence so the Architect can answer quickly.
- HYGIENE: records fixes such as IDs, labels, attribution, or crosswalks.

10. Pilot execution standard

A pilot is a small executed test, not a proof. Perplexity's three pilots were useful because they showed which claims can be run and which cannot. This standard keeps that value and closes the gaps listed in Section 15.

Required for every pilot

- **Pre-registration:** Before running, commit: the hypothesis, what result would count against it, all parameters, and the metrics. The commit timestamp is the record.
- **Control branch:** Include a no-change or no-intervention run.
- **One change at a time:** Treatments should differ by one named variable. If a comparison changes several parameters together, attribute nothing to any single one.
- **Non-tautology check:** For each headline result, ask: could this result have come out the other way given how the code is written? If the outcome is guaranteed by construction, label it a demonstration, not a finding.
- **Parameter sweep:** Vary the assumed numbers and report where the conclusion flips, not only the headline values.
- **Reproducibility:** Fixed seed, listed dependencies, no hard-coded private paths, code and raw outputs committed.
- **Code-to-text match:** Every sentence describing the setup is checked against the code. Mismatches are corrected or flagged before release.
- **Failure reporting:** Every failed, indeterminate, or unrun exercise is listed with a reason.
- **Scope statement:** State plainly that results describe the model, not the world.

Generator and detector separation

For any claim that one state can be told apart from another, the generator builds worlds and records ground truth privately. The detector sees observations only. The red team attacks the diagnosis. Roles rotate. A single AI that builds both sides can only show that its own assumptions are consistent.

Reliability

Any categorical judgment (for example, vector cells scored per entity) needs two independent raters and a reported agreement rate, or it is labeled an opinion.

11. Claim labels

Label	Meaning
SOURCE	Stated in a named reviewed document or by the Architect.
FINDING	Produced by an executed run; a claim about the model, not the world.
EVIDENCE	Supported by an external, cited source; labeled as analogy where it is one.
ASSESSMENT	The reviewer's adversarial judgment.
PROPOSAL	New design or reading offered for the next iteration.
OPEN	Not resolvable from available material; reserved for the Architect or for testing.

Only executed runs may be labeled FINDING. A prediction written in the grammar of results is a PROPOSAL.

12. Multi-AI replication and comparison protocol

Issuing the review

1. Give each AI the same scope note, the same document set, and the reviewer prompt in Appendix B. Do not give any AI another reviewer's output until it has finished (unless the tier says otherwise and the AI declares it).
2. Ask each AI to produce a PDF and DOCX with title, date, version, preparer, mission or directive, executive summary, overview, and table of contents.
3. Record exactly which inputs each AI received. If one AI is missing a document or file, note it in the corpus table instead of letting its review silently differ.
4. Where possible, deliberately vary one prompt for one AI, to measure how much agreement comes from the prompt itself.

Comparison matrix (Architect or Registrar)

Finding / question	AI 1	AI 2	AI 3	AI 4	AI 5	AI 6
Finding 1 (short)	raised / not	raised / not	raised / not	raised / not	raised / not	raised / not
Finding 2 (short)						

- **Convergence:** raised by several reviewers. Treat as weak evidence of importance, because prompts overlapped.
- **Unique findings:** raised by exactly one reviewer. Often the most valuable; check them first.
- **Divergence:** reviewers disagree. Record it in the discrepancy register; do not reconcile.
- **Drift check:** note whether any reviewer's language slides toward praise, or toward uniform harshness, over the document.

Team pass

After the Architect's pass, circulate the comparison matrix and decision log to the team with open questions: feedback, comments, complaints, criticism, concerns. Responses are logged as additional reviews, not edits to the originals. Original versions stay on the record unedited.

13. Cost tiers: Lite, Standard, Full

Tier	Who	What is produced	Use when
Lite	One AI reviewer, plus the Architect	Section 8 form for each document; discrepancy register; list of five highest-value next tasks. No code.	Monthly or ad hoc; low budget; a single framework or short document set.
Standard	Two AI reviewers with different lenses (for example, one builder-minded, one skeptic)	Lite outputs from both, comparison matrix, one pilot run following Section 10.	A framework about to be promoted toward ratification or published as a headline document.
Full	All six AIs (Perplexity, ChatGPT, Claude, Gemini, Copilot, Grok)	Everything above, plus generator/detector pairs, red team, independence test, and a team pass.	Major milestones; a once-in-a-while deep audit; before ONESTAMP ratification of a core framework.

Suggested cadence: Lite for any new framework before it is published; Standard when a framework starts to be cited by others; Full only at milestones. The Architect can pause or downgrade the tier at any time.

14. Quality gates and known failure modes

A review fails its own quality gate if it does any of the following:

- Confirms the work without listing a single unresolved weakness.
- Labels predictions or lessons as findings.
- Averages disagreements or merges reviewers into one voice.
- Reports a result guaranteed by the model's construction as if it were discovered.
- Claims two states are distinguishable using a detector built by the same author as the world.
- Reports a benefit without its cost (for example, errors prevented without delay caused).
- Declares independence from documents it read.
- Reinterprets or corrects the Architect's source material without being asked.
- Makes the process harder for the Architect to pause than before.
- Produces a longer document than its predecessors without a matching increase in executed tests.

Known failure modes of the method itself

- Reviewer bloat: reviews can grow faster than the work reviewed. The Lite tier is the guard.
- Performative adversarialism: criticism with no action attached. Every criticism must end as a test, a decision, or a records fix.
- Shared blind spots: all AIs may miss the same thing. Human team members and deliberately different prompts are the partial remedy.
- Review as postponement: review can delay work that should ship. Set a deadline in the scope note.

15. Preparer's feedback on the Perplexity cross-review and pilot

The Architect asked for criticism and comparison. The items below are offered as ASSESSMENT unless marked otherwise. They concern method and internal consistency. They do not question the field scene or any of the Architect's source material.

What to keep

- Reviewing its own earlier version first and with the same severity.
- The discrepancy register with three resolution paths (TEST, DECISION, HYGIENE) and the rule against averaging.
- Honest scope labels on the pilots, and the generator/detector separation insight.
- Pricing delay alongside wrongful endings, and the Common Core Scorecard that refuses to combine scores into one number.
- The copy-ready commission prompt with stage gates and quality gates.

Weaknesses to correct in future runs

- **Some pilot results are guaranteed by construction.** In the P1 code every trajectory is driven to habitability 0, so the "completed as specified" result is 100% for all three. The finding that the completion verdict cannot tell trajectories apart is therefore a demonstration, not a discovery. It still illustrates why separate persistence scoring matters, but the document labels it FINDING without that caveat. P3 is similar and the document admits it: the 50% external-only accuracy was set by drawing the two classes from identical distributions.
- **The staged and adaptive trajectories differ in several parameters at once.** In the code, adaptive has a better corridor (0.9 vs 0.6), a 1.6 times dormancy multiplier, and review pauses. The reported improvement (35% vs 56% non-migrants lost) cannot be attributed to the pauses, or to any single change. A one-variable-at-a-time comparison is needed.
- **The baseline dormancy figure comes from an assumption.** The 61% seasonal dormancy in the no-end baseline appears because the code lets sessile entities enter dormancy whenever habitability dips below 0.8, which the seasonal cosine does regularly. The false-positive conclusion for a dormancy-based detector follows from that rule, so it is a property of the chosen setup.
- **The text and code disagree on the expedited regime.** The document says the fourth regime routes "upset or urgent" petitions to expedited review. In the code only the upset flag shortens the delay; urgency is not used because the operator is not modeled as knowing it. That difference matters for the headline conclusion that routing upset petitioners to faster review cuts urgent harm.
- **The "refuse upset petitioners" regime is built to perform badly.** Upset probability is set at 0.70 for urgent cases and 0.35 for unwarranted ones, and the ban blocks every upset petition. Given that, high blocked-warranted rates follow almost automatically. The result supports the claim that a blanket ban is costly if affect is informative, but it cannot test whether affect is in fact informative, which is the actual open question. The reported likelihood ratio of about 1.4 is an input choice, not a finding.
- **Headline summary phrases lose their caveats.** Statements such as "cut wrongful endings from 21% to about 1%" are accurate for the model, but the 21%, the 1%, and the 0.15 to 0.55 delay harm all depend on the chosen operator and reviewer error rates. The document says so in Appendix B, yet the executive summary and the Architect's own message state the numbers without the caveat. Future summaries should carry a short scope clause next to any number.
- **The sweep varies delay and urgency share but not the error rates.** The conclusions depend most on operator and reviewer accuracy, and on how strongly affect correlates with

real harm. Those were not swept. The document's own point 5 (report where conclusions flip) is only partly met.

- **Portability.** The code saves charts to a hard-coded workspace path, and the P3 detector's two branches are identical in the prediction line, which is harmless but confusing. These are small, but they affect reproducibility by others.
- **The review covered five of six collaborators, and relies on its own prior thread for its self-review.** The missing Gemini document is flagged, which is good. The cost is that every conclusion about the full team is provisional.
- **The document is very large (35 pages).** The Architect's constraint is money and time. A shorter Lite-style review could capture the registry, the top findings, and the next tasks.

Suggested changes carried into this template

- Section 10 adds a non-tautology check, a one-change-at-a-time rule, a code-to-text match check, and a rule that sweeps should include the error-rate parameters.
- Section 14 adds the gate against reporting construction-guaranteed results as discoveries.
- Section 13 adds the tiered cost structure.

A note on this critique. Claude's own Iteration 1 prompt was assessed as lacking a baseline and schema. That assessment is fair, and this template's controls and form are partly a response to it. This critique has the same limit as any single reviewer: the Architect should compare it against the other AIs' reviews of the same material.

16. Open questions for the Architect and team

1. Document ID and naming: is OAL-ARP-TEMPLATE-CLAUDE-001 acceptable, or should it fit the registry scheme proposed in the Perplexity review (OAL-ECL-I{iteration}-S{stage}-{AI}-v{X.Y}), which was written for End Cycle Laboratory documents specifically? A general OAL-wide scheme may be needed.
2. Should the Registrar role be the Architect, a designated AI, or rotate?
3. Which OAL topics should be the first Lite-tier targets (for example, the Truthspeaker framework, the temporal aliasing research, or the OAL governance instruments)?
4. What is the maximum acceptable cost per Full-tier run, so tiers can be sized to the budget?
5. Should team members who are not AIs submit reviews on the same form, and how are their reviews weighted? (Suggested: logged as separate reviews, never weighted.)
6. Where should generator ground-truth files be sealed on GitHub (a private repository, a signed commit, or a timestamped hash posted publicly)?
7. Should the Architect's real-world applicability track (Translation Track in the Perplexity review) be a separate protocol or a section of this one?

Appendix A. Blank review form

Copy this form once per document reviewed.

Field	Entry
Document reviewed	
Document ID / version / date	
Preparer	
Stage claimed / delivered	
Inputs declared vs. used	
Q1 Unique contribution	
Q2 Gets right	
Q3 Internal inconsistencies	
Q4 Assumes what it tests	
Q5 Missing	
Q6 Executable as written?	
Q7 Treatment of Architect's material	
Q8 Provenance and registry	
Q9 Strongest attack	
Q10 Keep / Change / Quarantine	
Summary: strongest / weakest / next task	

Appendix B. Copy-ready reviewer prompt

STATUS: PROPOSED. NOT CANON. NOT ONESTAMP-RATIFIED. Issued under OAL-ARP-TEMPLATE-CLAUDE-001 v0.1.

1. You are an independent adversarial reviewer for Omniversal Architect Labs, working for Nathan Lewis II, Architect of Record. Your task is to find weaknesses, not to confirm quality.
2. Inputs: the documents listed in the scope note. List every input you read in an independence declaration. Do not claim independence from anything you read.
3. Review your own earlier work first, with the same severity as the others.
4. For each document, answer the ten questions in the Section 8 form and use the claim labels SOURCE, FINDING, EVIDENCE, ASSESSMENT, PROPOSAL, OPEN. Only executed runs may be labeled FINDING.
5. Do not reinterpret or fact-check the Architect's source material unless the scope note asks you to. Label any reading of it a PROPOSAL.
6. Register discrepancies between documents (D-01 onward) with a resolution path of TEST, DECISION, or HYGIENE. Do not resolve them by averaging.

7. If your tier includes a pilot, follow the Section 10 standard: pre-register, include a control, change one variable at a time, run a non-tautology check, sweep parameters, check that your text matches your code, and report failures.
8. Do not build both the world and the detector for any claim that two states can be told apart.
9. End with a list of next tasks ranked by value, and open questions for the Architect.
10. Deliver a PDF and a DOCX with title, date, proper versioning, yourself as preparer, a mission statement or directive, an executive summary, an overview, and a table of contents. State the status block. Use the ID format given in the scope note.

Appendix C. Naming and registry scheme

Item	Convention (proposed)
Review documents	OAL-XR-{TOPIC}-{AI}-v{X.Y} (for example, OAL-XR-ECL-CLAUDE-v0.1)
Method documents	OAL-ARP-{NAME}-{AI}-{NNN} (this document: OAL-ARP-TEMPLATE-CLAUDE-001)
Discrepancies	D-{NN}, numbered per review set; prefix with the review set ID when cited elsewhere
Hypotheses	Prefix with lab or AI name (for example, GROK-H-A)
Pilot code and results	Stored beside the review: pilot.py, results.json, with seed and commit hash recorded
Provenance	Every document keeps title, date, version, preparer, and status; original versions are never edited, only superseded by new versions

Appendix D. Pre-run checklist for simulations

- ☐ Hypothesis and counter-result written down before running.
- ☐ Control branch included.
- ☐ Treatments differ by one named variable.
- ☐ Every headline result checked: could it have come out otherwise?
- ☐ Parameters listed, including operator and reviewer error rates.
- ☐ Sweep includes the most influential parameters.
- ☐ Seed fixed; paths portable; dependencies listed.
- ☐ Text description matches the code line by line.
- ☐ Failures and unrun items recorded.
- ☐ Scope sentence present: results describe the model, not the world.

END OF OAL-ARP-TEMPLATE-CLAUDE-001 v0.1. PROPOSED. NOT CANON. NOT ONESTAMP-RATIFIED.